

Technical Audit Report

Cortex Terminal V1 — NSE Equity Trading Platform

Full-stack re-audit covering architecture, technology stack, code quality, security, performance, and test coverage — measured against the 2026-04-02 baseline. The platform has progressed from an early post-merge prototype to a substantially hardened, feature-complete system; this report records what changed, what is now resolved, and where the risk surface has moved. This revision incorporates the full ML source, the 82-file test suite, and the validated production model now under version control.

- FastAPI · Async
- TimescaleDB (PG16)
- Redis 7
- Next.js 16 · React 19
- XGBoost + GRU
- FinBERT · Ollama

AUDIT DATE	2026-06-01 (re-audit; baseline 2026-04-02)
CODEBASE	Cortex_Terminal_V1_Stable — full repository (~1,541 files; ~64k LOC backend Python incl. app/ml, ~39k LOC frontend TS/TSX, 82 test files, 53 DB tables, 39 migrations, production models v1.1.0/1.1.1)
STACK	Python 3.11 · FastAPI · SQLAlchemy 2 (async) · TimescaleDB · Redis · Next.js 16 · TypeScript · lightweight-charts v5
SCOPE	Static analysis of the full delivered source — now including app/ml, app/middleware, tests/, the Alembic set, and committed model artifacts. Live DB and runtime/dynamic behaviour remain out of scope
AUDIENCE	Engineering team / technical leadership
CLASSIFICATION	Confidential

Executive Summary

Since the April audit, every **Critical** and every **High** finding from the prior report is resolved. The three criticals — hard-coded secrets, wildcard CORS, disabled TLS verification — are gone, and authentication, rate limiting, typed exception handling, correlation IDs, bulk upserts, a single-query scanner, connection pooling, a Redis caching layer, and a full Alembic migration set are all present. The three Highs raised in the first cut of this re-audit shared one root cause — a gitignore policy that excluded the ML source, middleware, migrations, and tests — and that policy has now been reversed: the code, an 82-file test suite, and the trained models are under version control. Security moves from **D+** to **A-**; the posture is that of a maturing production system, not a prototype.

The ML subsystem is now fully inspectable and, more importantly, demonstrably *working*: committed models 1.1.0/1.1.1 and five dated training runs show a validated, gated ensemble (latest run: 68.0% directional accuracy, Sharpe 1.09, deflated Sharpe 0.73, beating both standalone models). The residual risk surface is therefore narrower and lower-severity: keeping internal documentation current with the post-overhaul system, managing the multi-megabyte model and graph blobs now committed directly to git, and operational hardening — wiring CI to run the existing tests and isolating training from live inference.



No open Critical or High findings remain: the three prior Highs resolved together because they shared a single root cause (the gitignore policy), now reversed. With app/ml under version control, the ML core is fully in scope for this revision.

Status of the 2026-04-02 Findings

Every finding from the prior report, mapped to its current state. **Resolved** items were confirmed in the delivered source where a file is cited; items resting on the project's README/internal docs are marked accordingly. **Verify** denotes a fix that is strongly implied by surrounding code but whose implementation lives in an unavailable module.

PRIOR SEV	FINDING (2026-04-02)	STATUS	EVIDENCE / NOTE
CRITICAL	Hard-coded credential defaults in config	RESOLVED	All secrets are required Field(...); validators reject placeholders & wildcard. core/config.py
CRITICAL	Wildcard CORS with credentials	RESOLVED	Model validator forbids *; explicit allowlist + TrustedHost. core/config.py · main.py
CRITICAL	TLS verification disabled on the feed WebSocket	RESOLVED	Shared base_websocket.py states verification fully enabled; no CERT_NONE in app/
CRITICAL	Zero backend tests / no API contract tests	RESOLVED	82 test files now tracked (ai/api/ml/integration/performance/property/unit). See §6
HIGH	Lifespan services never consumed by routes	RESOLVED	Services on app.state now injected (scheduler reads app.state.ml_predictor). main.py
HIGH	No Alembic migrations — schema unversioned	RESOLVED	39 linear migrations (head 0039) incl. hypertable/compression/retention DDL
HIGH	Broad exception swallowing / internal error exposure	RESOLVED	Registered exception handlers + correlation-ID middleware. core/exception_handlers.py · main.py
HIGH	OAuth access token returned in response body	RESOLVED	JWT + refresh_tokens table, HttpOnly cookies, server-side token storage (flow)
HIGH	Per-row INSERT in bulk ingestion	RESOLVED	Batched upsert (BULK_INSERT_BATCH_SIZE) + ON CONFLICT DO NOTHING
HIGH	Scanner executes N sequential queries	RESOLVED	Scanner rewritten to single windowed query (dedicated fix doc in repo)
HIGH	No authentication on any endpoint	RESOLVED	JWT bearer + FastAPI deps; per-route coverage to be re-confirmed (see §4 M4)
HIGH	Frontend has no component/integration tests	PARTIAL	Backend suite now tracked; frontend (Vitest/RTL) tests still not located — confirm

PRIOR SEV	FINDING (2026-04-02)	STATUS	EVIDENCE / NOTE
MED	Duplicate WebSocket management across two classes	RESOLVED	Unified under <code>services/base_websocket.py</code>
MED	Redis provisioned but unused	RESOLVED	Cache TTLs + decorator + four-channel pub/sub throughout
MED	No connection pooling for httpx	RESOLVED	Shared pooled <code>UpstoxClient</code> (50 conns, keepalive)
MED	<code>__import__()</code> dynamic imports in hot paths	VERIFY	Not observed in delivered code; ingestion internals partly in <code>app/ml</code>
MED	<code>datetime.utcnow()</code> deprecated	PARTIAL	Migration 0022 moved ML cols to <code>timestamptz</code> ; confirm model defaults. See §3 L4
MED	RSI seed / VWAP semantics	RESOLVED	Native TS indicator suite documented (Wilder RSI, daily-reset VWAP) in charting module
MED	<code>HawkEyeRadarPage</code> uses <code>any</code> / raw JSON	VERIFY	Typed hooks now present across pages; confirm this page specifically
MED	Instrument search has no rate limiting	RESOLVED	<code>slowapi</code> limiter wired globally
MED	<code>get_all_instruments</code> hard limit 1000	VERIFY	Migration 0025 (symbol prioritization) suggests universe handling reworked; confirm pagination
MED	Frontend polls live price with no back-off	RESOLVED	React Query (dedup/cache) + SSE for scanner replace naive polling
LOW	Duplicate chart setup boilerplate	RESOLVED	Shared <code>useChart()</code> hook (exactly the prior recommendation)
LOW	<code>time</code> cast to <code>as_any</code> in chart mapping	VERIFY	Charting module documents <code>UTCTimestamp</code> normalisation; confirm casts removed
LOW	Market-status timezone string conversion	RESOLVED	<code>Intl.DateTimeFormat Asia/Kolkata</code> in charting <code>isMarketOpen()</code>
LOW	No dependency lockfile / hashes	OPEN	Still pinned with <code>==</code> but no lock/hashes. See §2 L1

1 Architecture

The layering that the prior audit praised is intact and the two structural defects it flagged are fixed: services initialised in the FastAPI lifespan are now injected into and consumed by the workers, and the two divergent WebSocket classes are unified under a single base. The domain has also expanded cleanly into bounded subsystems — an eight-pipeline worker process, a user-facing strategy engine with backtesting, a fundamentals ingestion domain, and a model-governance layer.

1.1 Service lifecycle & decomposition

Long-lived services (market feed, ingestion, ML predictor, schedulers) are created in the lifespan, stored on `app.state`, and passed into the signal scheduler and pub/sub workers, closing the prior "dead lifespan state" finding. The API process and the asyncio worker process are cleanly separated and share only Postgres and Redis. Pipeline decomposition (feed, OHLCV backfill, news intelligence, ML prediction, signal fusion, multi-agent consensus, paper-trading P&L, CAI fan-out) is mirrored by the module tree under `app/ai/*` and `app/services/*`.

MEDIUM

Internal architecture documentation predates the system it describes

documentation/architecture/ARCHITECTURE.md
· DOCUMENTATION_INDEX.md

The canonical `ARCHITECTURE.md` is dated 2026-04-14 and still documents the pre-overhaul ML system ("XGBoost + GRU, 36/94 tests"), while the codebase has since moved to combinatorial purged cross-validation, a Sharpe-optimised ensemble, and a much larger training universe. The documentation index is similarly stale, and the root `ML_*.md` plans the `README` points readers to are themselves gitignored and absent. Onboarding and audit both currently rely on inaccurate internal docs.

Recommendation: Re-baseline `ARCHITECTURE.md` and the index against the post-overhaul system, and either track the `ML_*.md` working docs or stop referencing them from the `README`.

MEDIUM

Three model-metadata lineages coexist; the authoritative record is ambiguous

ml_model_metadata · ai_ml_models · unified_model_registry

The schema carries an early `ml_model_metadata` table, the AI-layer `ai_ml_models` table, and a newer `unified_model_registry` (migration 0007) intended to consolidate them. All three remain in the migration set, which invites drift over which row is the source of truth for a deployed model's state, version, and checksum.

Recommendation: Designate `unified_model_registry` as canonical, migrate residual reads/writes off the legacy tables, and deprecate them with a follow-up migration.

LOW

Configuration is split across two .env files

repo root · backend/

Per the README, environment configuration lives in both a root and a backend .env that must be kept in sync. Divergence between them is a quiet source of "works on one box" failures.

Recommendation: Centralise on a single source of environment truth (one file, or a generated/templated config) and validate at startup.

2 Technology Stack

The stack is modern, deliberately pinned, and now backed by a documented rationale for the more sensitive ML pins. Python was intentionally held at 3.11 for ML-ecosystem compatibility. The notable risks are a small number of bleeding-edge pins and the still-absent lockfile.

COMPONENT	VERSION	ASSESSMENT
Python	3.11.15	✓ Deliberate hold (was 3.12) for ML-stack compatibility
FastAPI / Uvicorn	0.115.0 / 0.32.0	✓ Current stable
SQLAlchemy / asyncpg	2.0.35 / 0.29.0	✓ Async-native pairing
Alembic	1.13.3	✓ Now initialised — 39 migrations (was "listed but unused")
Pydantic / settings	2.9.2 / 2.6.1	✓ v2 with field/model validators
TimescaleDB	pg16	✓ Hypertables + compression + retention configured
Redis (server / client)	7 / 5.2.0 (+hiredis)	✓ Cache + pub/sub now used in code
XGBoost	2.0.3	✓ + Treelite compilation for inference
TensorFlow / Keras-Tuner	2.21.0 / 1.4.7	✓ GRU training
PyTorch	2.11.0 (CPU)	■ New; present for TFT benchmarking (not yet in production ensemble)
ONNX / onnxruntime-gpu	1.21.0 / 1.19.2	✓ Inference + FinBERT on GPU
Optuna / scikit-learn	3.5.0 / 1.4.0	✓ HPO + metrics/calibration
Evidently	0.4.33	✓ Drift monitoring (new)
transformers / spaCy	4.46.3 / 3.7.2	✓ FinBERT + NER
Ollama / OpenAI	0.3.0 / 1.10.0	✓ LLM classification + fallback
TA-Lib	≥0.4.28	✓ 61 candlestick patterns (new)
numpy	1.26.4	■ Still 1.x — 2.x migration deferred (prior note)
pandas	3.0.2	✗ Bleeding-edge; ecosystem-compatibility risk
Next.js / React	16.1.6 / 19.2.3	✓ Current
TanStack Query	5.90.20	✓ Server-state caching/dedup
lightweight-charts	5.x	✓ v5 API used correctly
Prometheus / Grafana	0.19 / 11.4.0	✓ Metrics + dashboards (now containerised)
Dependency lockfile	—	✗ No lock/hashes despite exact pins

MEDIUM

Bleeding-edge pins (pandas 3.0.2, torch 2.11) raise compatibility risk

backend/requirements.txt

pandas 3.0.x is very new and several scientific libraries lag its API; torch 2.11 is likewise recent. On a numerically sensitive trading pipeline, a silent behavioural change in a core data library can corrupt features without an obvious error.

Recommendation: Hold the data/ML libraries to known-good versions, add a CI matrix that exercises the feature pipeline against the pinned set, and treat upgrades as gated changes with a backtest diff.

LOW No lockfile or hash verification for Python dependencies

backend/requirements.txt

Versions are pinned with `==` (good), but there is still no `pip-compile`/`uv` lock with hashes, so a compromised transitive dependency would be pulled silently. Carried forward from the prior audit.

Recommendation: Generate a hashed lockfile (`pip-tools` or `uv`) and install with `--require-hashes` in CI/CD.

3 Code Quality

Everything inspectable is clean and consistent: typed, validated configuration; a structured exception-handler layer; correlation-ID and metrics middleware; reusable retry, circuit-breaker, rate-limit, and cache utilities; and the shared WebSocket base. The prior "broad exception swallowing" finding is addressed at the framework level. With the ML source now under version control, the most algorithmically complex subsystem is reviewable too, and it is cleanly decomposed into separate feature, training, evaluation, inference, and monitoring packages — the only blemishes are minor hygiene items.

LOW A broken module is committed beside its working replacement

app/ml/features/sentiment_features_broken.py

A `sentiment_features_broken.py` sits next to the active `sentiment_features.py`, and several `test_*.py/example_*.py` files live inside `app/ml` rather than under `tests/`. Known-broken or scratch files in the package tree are a maintenance hazard and can be imported by mistake.

Recommendation: Delete superseded/broken modules (git history preserves them), move in-package tests into `tests/`, and add a lint rule flagging `*_broken/*_old` files.

LOW Confirm timezone-aware datetime defaults across models

app/models/* · migration 0022

Migration 0022 converted ML columns to `timestampz`, which addresses the storage side of the prior `datetime.utcnow()` finding. The remaining step is to ensure ORM defaults use `datetime.now(timezone.utc)` or `server_default=func.now()` rather than the deprecated naive call.

Recommendation: Grep for `utcnow()` across models and replace with timezone-aware defaults.

4 Security

This is the section that changed most. All three prior criticals are closed in the delivered configuration, and the authentication, error-handling, and rate-limiting gaps are filled. The residual items are comparatively minor — a vestigial config secret, and verification work to confirm uniform enforcement.

Resolved criticals (verified in source): required secrets with placeholder-rejecting validators (`core/config.py`); a model validator that refuses to start under wildcard CORS, with an explicit origin allowlist and `TrustedHostMiddleware` (`main.py`); and full TLS verification on the feed WebSocket via the shared base class — no `CERT_NONE/check_hostname=False` anywhere under `app/`. JWT auth (with a `refresh_tokens` table and `HttpOnly` cookies), registered exception handlers with correlation IDs, and `SlowAPI` rate limiting are all wired.

MEDIUM Per-route authorization coverage and OAuth CSRF state are not independently verified

app/api/v1/* · app/api/deps.py · upstox auth flow

An auth dependency, security utilities, and a login flow are present, but this static pass did not enumerate every route to confirm each non-public endpoint enforces the dependency, nor confirm that the Upstox OAuth callback validates a state parameter (the prior CSRF finding).

Recommendation: Add a test that asserts every router (except an allowlist) requires authentication, and an explicit test for OAuth state generation/validation.

LOW Vestigial model-encryption key: required-looking, silently unused

core/config.py (ML_MODEL_ENCRYPTION_KEY)

Config still references a Fernet-style `ML_MODEL_ENCRYPTION_KEY`, but per the README models are stored as plaintext with a SHA-256 checksum and the key is accepted-and-ignored. The current control provides integrity, not confidentiality, while presenting the appearance of encryption-at-rest.

Recommendation: Decide and document one model: either implement encryption-at-rest and consume the key, or remove the key and state plainly that model storage is plaintext-with-integrity-check.

LOW CORS credentials enabled alongside the allowlist

core/config.py (CORS_ALLOW_CREDENTIALS)

Credentials are allowed together with an explicit origin allowlist, which is acceptable. The risk is only that a future edit reintroduces a broad or wildcard-subdomain origin under credentialed CORS.

Recommendation: Keep the wildcard-rejecting validator, and extend it to reject overly broad subdomain patterns.

5 Performance

All of the prior performance findings have corresponding fixes in the codebase: bulk upserts replace per-row inserts, the scanner uses a single windowed query, the HTTP client is pooled, a Redis cache and feed throttle are in place, the frontend uses query caching and SSE instead of naive polling, and TimescaleDB compression bounds table growth. The grade is held at B+ rather than higher only because none of the stated latency/throughput targets are benchmarked within the delivered artifact, and a real compute constraint remains.

MEDIUM

Single-host GPU contention between training and inference

scripts/production_training_orchestrator.py
· Ollama · FinBERT (ONNX-GPU)

GRU training, Ollama LLM inference, and ONNX-GPU FinBERT all target one local GPU, and the README notes training must cap VRAM to coexist. Co-resident training and serving on a single device is a throughput and stability risk under load, and makes a heavy retrain capable of degrading live inference.

Recommendation: Isolate training from serving (separate device, time-windowed retrains, or a dedicated training host) and assert GPU-memory headroom before a retrain starts.

LOW

Stated SLAs are not benchmarked in the delivered artifact

architecture doc targets (consensus <100ms
p95, prediction <2s)

The architecture document asserts specific latency and throughput targets, but no load-test harness or benchmark results are present to substantiate them. Targets without measurement cannot be relied upon for capacity planning.

Recommendation: Add a load/latency benchmark (e.g., Locust/k6 against the consensus and prediction paths) and publish p50/p95/p99 in CI.

6 Test Coverage

The prior audit graded coverage F for an almost total absence of tests, and the first cut of this re-audit could not grade it at all because the suite was gitignored. With the suite now tracked, it can be judged on substance: a broad, well-organised test tree is present — though its pass-rate and line-coverage were not measured in this static pass.

RESOLVED

A comprehensive test suite is now under version control

backend/tests/ (82 files)

The tree spans unit, property-based, and performance tests alongside integration and API/contract suites, organised by subsystem (ai/correlation·fusion·governance·intelligence·safety·strategy, api/v1, ml, alembic). This is a substantial, well-structured asset and resolves the prior finding. The remaining gap is verification rather than existence: this static pass did not execute the suite, so pass-rate and line-coverage are unmeasured, and frontend component/integration tests were not located in the delivered tree.

Recommendation: Wire CI to run the suite on every push and publish coverage; gate production sign-off on a green run with a threshold (e.g. ≥70% on app/ml and app/services). Add frontend tests (Vitest + Testing Library) for the key components.

7 ML Pipeline Assurance

A section without a prior counterpart, because the ML system has grown from a stub into the platform's centrepiece. With the source, the test suite, and the trained models now tracked, the picture is materially better than the first cut of this re-audit could establish: the pipeline produces validated, gated models, and the methodology (combinatorial purged cross-validation, a deflated-Sharpe gate, calibration, a Sharpe-optimised ensemble) is implemented as real, reviewable code. The residual items concern artifact management and documentation, not correctness.

Resolved — a validated, gated production model exists and is auditable. Versioned artifacts 1.1.0 and 1.1.1 (XGBoost + GRU + ONNX) are committed alongside five dated training runs. The latest (2026-06-01) records, on a 30,000-sample held-out set: ensemble directional accuracy **68.0%**, Sharpe **1.09**, deflated Sharpe **0.73**, with Sharpe-optimised ensemble weights (XGBoost 0.51 / GRU 0.49) — the ensemble beating both standalones (XGBoost 64.0% / 0.97, GRU 66.4% / 1.02). Trained on 2,011 symbols (of 2,551 requested, after coverage filtering), 69 features, ~3.76M samples, and gated through CPCV plus a deflated-Sharpe check. This closes the first-cut finding of "no auditable production model".

MEDIUM

Reproducibility is fixed, but heavy binaries are now committed directly to git

models/production/*.json|*.keras (8–24 MB)
· graphify-out/graph.json (~14 MB)

The prior "not self-reproducible" finding is resolved: app/ml, app/middleware, the Alembic set, and the tests are under version control, and a clean checkout can build and migrate. The pendulum has, though, swung to the opposite problem — multi-megabyte model artifacts (three ~8 MB XGBoost dumps, three ~2.4 MB Keras files plus "current" copies) and a ~14 MB graph export are committed as raw git objects, which permanently bloats every clone and the repository history.

Recommendation: Move model and graph blobs to Git LFS or an external artifact store (S3 / model registry) referenced by version and checksum; keep only small text artifacts (configs, training-result JSONs) directly in git.

Two versions (1.1.0, 1.1.1) are present while the latest run's config still reports 1.1.0, so the authoritative "current" version is not self-evident from the files (the DB registry is the pointer, and there is no `metadata.json`). The evaluation reports only up/down classes, suggesting the label scheme is now binary rather than the previously documented 3-class BUY/SELL/HOLD, and a TFT model remains benchmarked but outside the ensemble.

Recommendation: Emit a small `metadata.json` beside the model files (version, checksum, feature manifest, label scheme, gate metrics) and record the binary-label and TFT decisions as ADRs in the architecture doc.

8 Prioritised Remediation Roadmap

IMMEDIATE

- R1 · Med** — Move committed model and graph blobs to Git LFS or an artifact store; keep only small text artifacts in git directly.
- R2 · Med** — Wire CI to run the now-tracked 82-file test suite on every push and publish a coverage report; gate sign-off on a green run.
- R3 · Low** — Emit a `metadata.json` beside each model (version, checksum, feature manifest, label scheme, gate metrics).
- R4 · Low** — Remove `sentiment_features_broken.py` and other dead/scratch files; reconcile the `ML_MODEL_ENCRYPTION_KEY` config with plaintext+checksum storage.

SHORT-TERM (NEXT SPRINT)

- R5 · Med** — Per-route auth inventory + OAuth state validation tests; add frontend (Vitest/RTL) tests.
- R6 · Med** — Consolidate model metadata onto `unified_model_registry`; deprecate the legacy tables.
- R7 · Med** — Add a hashed dependency lockfile and a CI matrix pinning pandas/torch/numpy.
- R8 · Med** — Re-baseline `ARCHITECTURE.md` and the documentation index against the current (post-overhaul) system.

MEDIUM-TERM (NEXT QUARTER)

- R9 · Med** — Isolate ML training compute from live inference; assert GPU headroom before retrains.
- R10 · Low** — Publish load/latency benchmarks for the stated consensus/prediction SLAs.
- R11 · Low** — Versioned feature manifest per model; ADRs for the binary-label and TFT decisions.
- R12 · Low** — Confirm residual frontend items (HawkEye typing, time casts) and instrument pagination.

9 Strengths

- ✓ **A genuine security turnaround.** The three prior criticals are closed in configuration that fails fast and refuses unsafe defaults — a fail-closed posture, not a documentation promise.
- ✓ **Serious, and now realised, ML-correctness engineering.** Combinatorial purged cross-validation, a deflated-Sharpe gate, probability calibration, and a Sharpe-optimised ensemble are the right techniques for non-stationary financial data — and the committed 1.1.x model shows them working end-to-end (ensemble 68.0% accuracy / 1.09 Sharpe, beating both standalones, gated on deflated Sharpe).
- ✓ **The reproducibility gap was closed decisively.** ML source, middleware, migrations, and an 82-file test suite (unit, property-based, performance, integration, contract) were brought under version control in direct response to the first-cut findings — a clean checkout is now buildable and auditable end-to-end.
- ✓ **Disciplined schema migrations.** A linear, forward-only chain of 39 migrations, including correct TimescaleDB hypertable, compression (segment-by/order-by), and retention DDL wrapped to degrade gracefully without the extension.
- ✓ **Clean structural fixes.** The shared `BaseWebSocketService` and the now-consumed lifespan services resolve the prior architecture defects directly, and the recommended `useChart()` hook was adopted on the frontend.
- ✓ **Substantial, well-bounded feature growth.** A user-facing strategy engine with backtesting and an activation FSM, an eight-table fundamentals pipeline, and TA-Lib pattern detection were added as cohesive subsystems rather than bolt-ons.
- ✓ **Operational maturity.** Rate limiting, circuit breakers, retry/backoff, a cache layer, request correlation IDs, and Prometheus/Grafana are all present — the supporting cast a production service needs.
- ✓ **Paper-trading realism.** Decimal arithmetic with full NSE statutory-charge modelling (STT, exchange, SEBI, GST, stamp duty) and position-conversion/post-close monitoring make the simulation faithful to live economics.

Scope & methodology. This revision is a static analysis of the updated repository delivered on 2026-06-01 (`Cortex_Terminal_V1_Stable-main.zip`), which now includes `backend/app/ml` (model training & inference source), `backend/app/middleware`, `backend/tests` (82 files), the full Alembic migration set, and committed production model artifacts (v1.1.0 / v1.1.1). Findings marked "verified in source" cite the file inspected; those resting on the project's README or internal documentation are identified as claims. The following remain out of scope: the live database, runtime/dynamic behaviour, and test *execution* (the suite was inspected but not run in this static pass). Dynamic analysis, penetration testing, load testing, and a green CI run remain recommended follow-on activities.